

[[musttail]]-ling Our Way to a Faster Python Interpreter and New JIT Compiler

KEN JIN OOI

What is an interpreter exactly?

What is an interpreter exactly?

LOAD
LOAD
ADD_INT

X + X

pc = 0

stack = []

What is an interpreter exactly?

LOAD
LOAD
ADD_INT

X + X

pc = 0
stack = []



What is an interpreter exactly?

LOAD
LOAD
ADD_INT

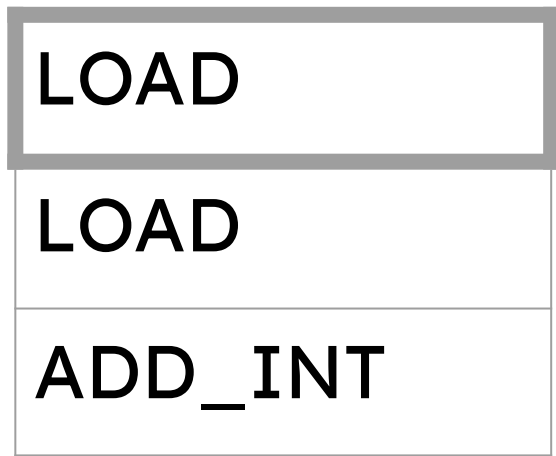
$X + X$

pc = 0

stack = []



What is an interpreter exactly?



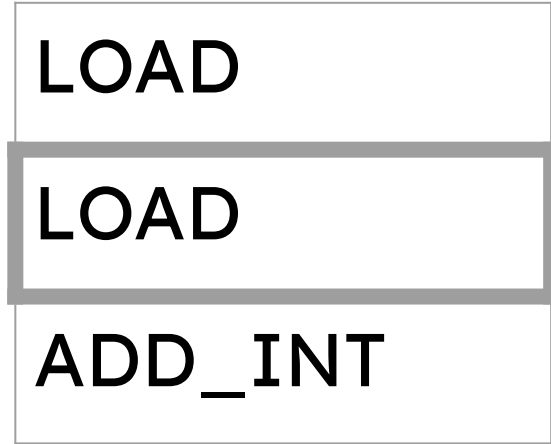
$x + x$

pc = 0

stack = [x]



What is an interpreter exactly?



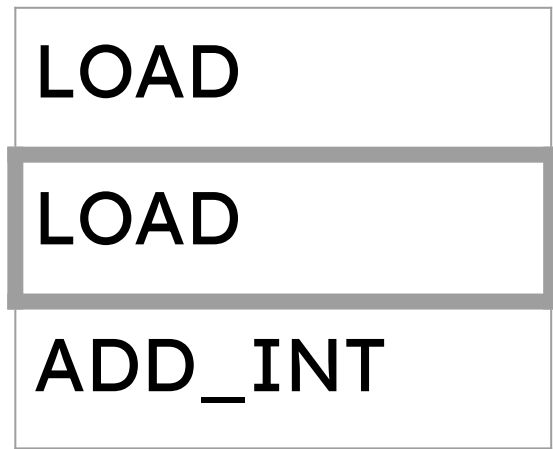
$x + x$

pc = 1

stack = [x]



What is an interpreter exactly?



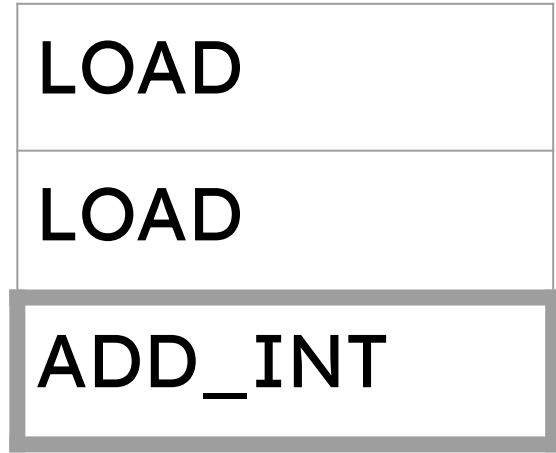
$x + x$

pc = 1

stack = [x, x]



What is an interpreter exactly?



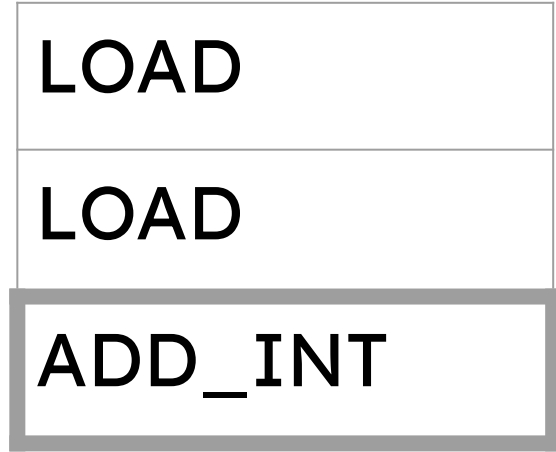
$x + x$

pc = 2

stack = [x, x]



What is an interpreter exactly?

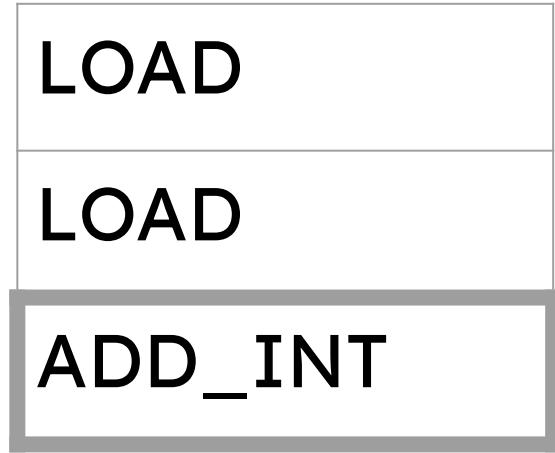


$X + X$

pc = 2
stack = []



What is an interpreter exactly?



$x + x$

pc = 2

stack = [x+x]



Why Should I Care?

Interpreter-like constructs are found everywhere:

- Language interpreters
- Parsers
- Serializers/deserializers
- Regex engines
- Almost any state machines

This is a common construct!

The Traditional Interpreter

Switch-case

```
switch (*ip) {  
    case INSTRUCTION_1:  
        // Subroutines.  
        ip++;  
        break;  
    case INSTRUCTION_2:  
        // Subroutines.  
        ip++;  
        break;  
}
```

Let's look at a popular interpreter: Python
(CPython)

Python before: 🐌

Python before: 🐌

**Why Python Loops Are "Slow":
A Gentle Introduction to Interpretation,
Bytecode, and Fast Code**

2025
PYOHIO



Tyler Riccio

Sponsored by:

 Intellovations

 python SOFTWARE FOUNDATION

Produced by:
**NEXT DAY
VIDEO**

Why Python Loops Are "Slow": A Gentle Introduction to Interpretation, Bytecode, and Fast Code



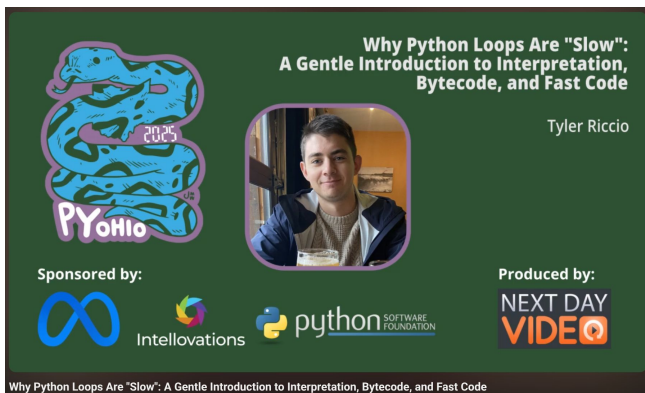
Miguel Grinberg

<https://blog.miguelgrinberg.com/post/python-3-14-is-...>

Python 3.14 Is Here. How Fast Is It?

8 Oct 2025 — These results also show that version 3.11 is the point at which **Python** versions moved from being "very **slow**" to "not so **slow**". One more detail ... [Read more](#)

Python before: 🐌



Python vs Rust for AI SaaS: A Balanced Look

★ This title was summarized by AI from the post below.



Igor Benav

6mo

...

Python gets criticized for being slow. Benchmarks show Rust and C++ running circles around it. Go handles thousands more requests per second. The critics aren't wrong about raw performance numbers (ignoring the fact that languages can't actually be slow or fast).



Miguel Grinberg

<https://blog.miguelgrinberg.com/post/python-3-14-is-...>

Python 3.14 Is Here. How Fast Is It?

8 Oct 2025 — These results also show that version 3.11 is the point at which **Python** versions moved from being "very **slow**" to "not so **slow**". One more detail ... [Read more](#)

Python before:



2025

Why Python Loops Are "Slow": A Gentle Introduction to Interpretation, Bytecode, and Fast Code

Tyler Riccio



Sponsored by:



Produced by:



Why Python Loops Are "Slow": A Gentle Introduction to Interpretation, Bytecode, and Fast Code

Python vs Rust for AI SaaS: A Balanced Look

✦ This title was summarized by AI from the post below.



Igor Benav

6mo

...

Python gets criticized for being slow. Benchmarks show Rust and C++ running circles around it. Go handles thousands more requests per second. The critics aren't wrong about raw performance numbers (ignoring the fact that languages can't actually be slow or fast).



Miguel Grinberg

<https://blog.miguelgrinberg.com/post/python-3-14-is-...>

Python 3.14 Is Here. How Fast Is It?

8 Oct 2025 — These results also show that version 3.11 is the point at which **Python** versions moved from being "very **slow**" to "not so **slow**". One more detail ... [Read more](#)

Python before:



Why Python Loops Are "Slow": A Gentle Introduction to Interpretation, Bytecode, and Fast Code

Tyler Riccio

Sponsored by:

Produced by:

Why Python Loops Are "Slow": A Gentle Introduction to Interpretation, Bytecode, and Fast Code

Why Python is Slow: Looking Under the Hood

Fri 09 May 2014

We've all heard it before: Python is slow.

When I teach courses on Python for scientific computing, I make this point [very early](#) in the course, and tell the students why: it boils down to Python being a dynamically typed, interpreted language, where values are stored not in dense buffers but in scattered objects. And then I talk about how to get around this by using NumPy, SciPy, and related tools for vectorization of operations and calling into compiled code, and go on from there.

But I realized something recently, despite the relative accuracy of the above statements, the words "dynamically typed-interpreted-buffers-vectorization-compiled" probably mean very little to somebody attending an intro programming seminar. The jargon does little to enlighten people about what's actually going on "under the hood", so to speak.

So I decided I would write this post, and dive into the details that I usually gloss over. Along the way, we'll take a look at using Python's standard library to introspect the goings-on of CPython itself. So whether you're a novice or experienced programmer, I hope you'll learn something from the following exploration.

Why Python is Slow

Python is slower than Fortran and C for a variety of reasons:

Python vs Rust for AI SaaS: A Balanced Look

★ This title was summarized by AI from the post below.



Igor Benav

6mo

...

Python gets criticized for being slow. Benchmarks show Rust and C++ running circles around it. Go handles thousands more requests per second. The critics aren't wrong about raw performance numbers (ignoring the fact that languages can't actually be slow or fast).

And it clearly matters!



Miguel Grinberg

<https://blog.miguelgrinberg.com/post/python-3-14-is-...>

Python 3.14 Is Here. How Fast Is It?

8 Oct 2025 — These results also show that version 3.11 is the point at which **Python** versions moved from being "very **slow**" to "not so **slow**". One more detail ... [Read more](#)

Why Python Loops Are "Slow": A Gentle Introduction to Interpretation, Bytecode, and Fast Code

Tyler Riccio

Sponsored by:

Produced by:

Why Python Loops Are "Slow": A Gentle Introduction to Interpretation, Bytecode, and Fast Code

Why Python is Slow: Looking Under the Hood

Fri 09 May 2014

We've all heard it before: Python is slow.

When I teach courses on Python for scientific computing, I make this point [very early](#) in the course, and tell the students why: it boils down to Python being a dynamically typed, interpreted language, where values are stored not in dense buffers but in scattered objects. And then I talk about how to get around this by using NumPy, SciPy, and related tools for vectorization of operations and calling into compiled code, and go on from there.

But I realized something recently, despite the relative accuracy of the above statements, the words "dynamically typed-interpreted-buffers-vectorization-compiled" probably mean very little to somebody attending an intro programming seminar. The jargon does little to enlighten people about what's actually going on "under the hood", so to speak.

So I decided I would write this post, and dive into the details that I usually gloss over. Along the way, we'll take a look at using Python's standard library to introspect the goings-on of CPython itself. So whether you're a novice or experienced programmer, I hope you'll learn something from the following exploration.

Why Python is Slow

Python is slower than Fortran and C for a variety of reasons:

Python after: 🐌

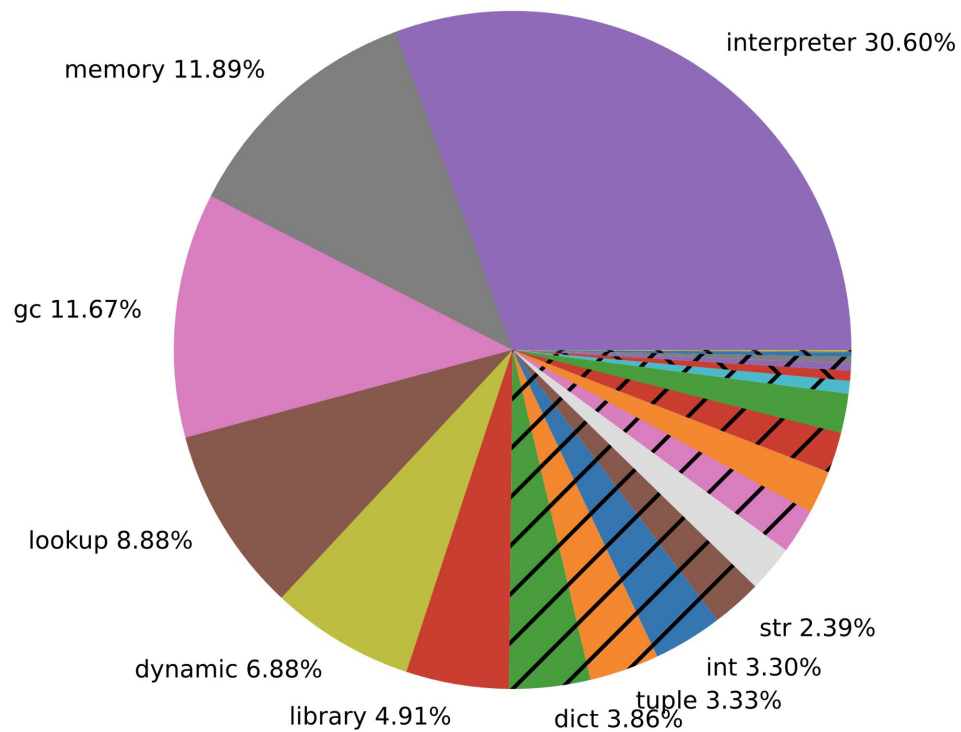
Python after: 🐌

Python after: 🐌++

How?

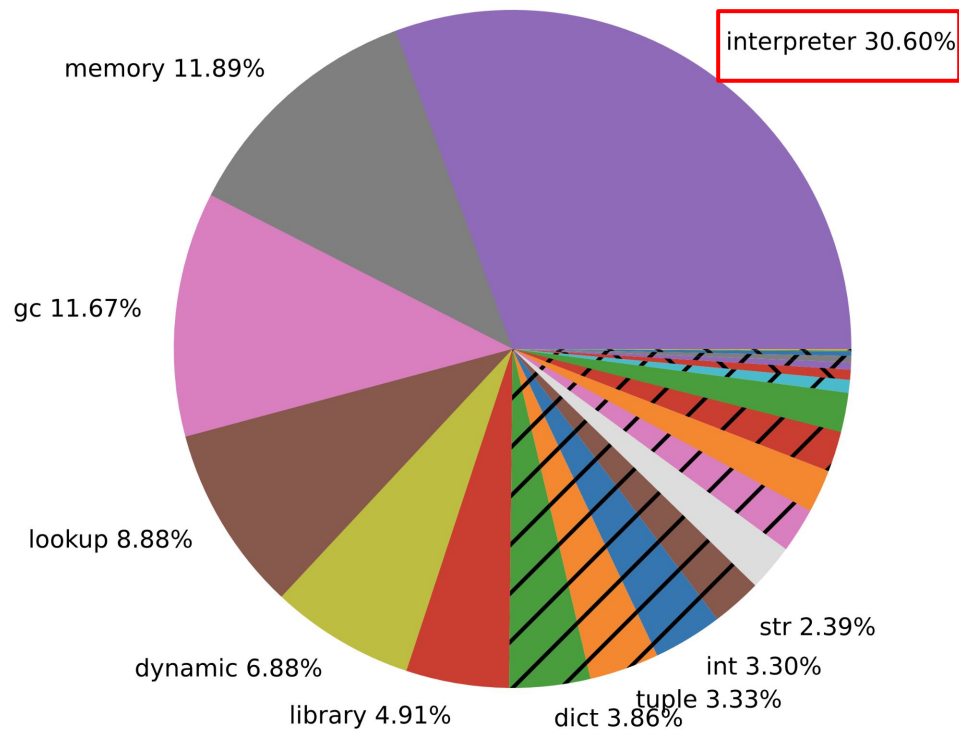
1. A more efficient interpreter
2. A new JIT compiler

How?



**historical data from 2025 from
our benchmark suite, prior to
this work*

How?



**historical data from 2025 from
our benchmark suite, prior to
this work*

3.14-15: A `[[musttail]]` Interpreter

The Traditional Interpreter

Switch-case

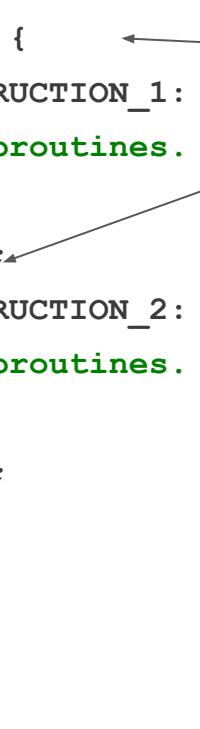
```
switch (*ip) {  
    case INSTRUCTION_1:  
        // Subroutines.  
        ip++;  
        break;  
    case INSTRUCTION_2:  
        // Subroutines.  
        ip++;  
        break;  
}
```

The Traditional Interpreter

Switch-case

```
switch (*ip) {  
    case INSTRUCTION_1:  
        // Subroutines.  
        ip++;  
        break;  
    case INSTRUCTION_2:  
        // Subroutines.  
        ip++;  
        break;  
}
```

Two jumps in a naive compiler
per instruction



The Traditional Interpreter

Switch-case

```
switch (*ip) {  
    case INSTRUCTION_1:  
        // Subroutines.  
        ip++;  
        break;  
    case INSTRUCTION_2:  
        // Subroutines.  
        ip++;  
        break;  
}
```

One jump, but many targets.
Bad for old* CPUs' branch
predictors.

*Note: doesn't matter much for newer CPUs, if interested, search "ITTAGE indirect branch predictor"

The current interpreter

CPython currently implements its interpreter using gotos/switch-cases:

```
void *DISPATCH_TABLE = {&&INSTRUCTION_1, &&INSTRUCTION_2,  
...};
```

```
goto *DISPATCH_TABLE[*ip];  
INSTRUCTION_1:  
    // Subroutines.  
    ip++;  
    goto *DISPATCH_TABLE[*ip];  
INSTRUCTION_2:  
    // Subroutines.  
    ip++;  
    goto *DISPATCH_TABLE[*ip];  
...
```

The current interpreter

CPython currently implements its interpreter using gotos/switch-cases:

```
void *DISPATCH_TABLE = {&&INSTRUCTION_1, &&INSTRUCTION_2,  
...};
```

```
goto *DISPATCH_TABLE[*ip];  
INSTRUCTION_1:  
    // Subroutines.  
    ip++;  
    goto *DISPATCH_TABLE[*ip];  
INSTRUCTION_2:  
    // Subroutines.  
    ip++;  
    goto *DISPATCH_TABLE[*ip];  
...
```

The current interpreter

CPython currently implements its interpreter using gotos/switch-cases:

```
void *DISPATCH_TABLE = {&&INSTRUCTION_1, &&INSTRUCTION_2,  
...};
```

```
goto *DISPATCH_TABLE[*ip];  
INSTRUCTION_1:  
    // Subroutines.  
    ip++;  
    goto *DISPATCH_TABLE[*ip];  
INSTRUCTION_2:  
    // Subroutines.  
    ip++;  
    goto *DISPATCH_TABLE[*ip];  
...
```

The current interpreter

CPython currently implements its interpreter using gotos/switch-cases:

```
void *DISPATCH_TABLE = {&&INSTRUCTION_1, &&INSTRUCTION_2,  
...};
```

```
goto *DISPATCH_TABLE[*ip];  
INSTRUCTION_1:  
    // Subroutines.  
    ip++;  
    goto *DISPATCH_TABLE[*ip];  
INSTRUCTION_2:  
    // Subroutines.  
    ip++;  
    goto *DISPATCH_TABLE[*ip];  
...
```

The current interpreter

CPython currently implements its interpreter using gotos/switch-cases:

```
void *DISPATCH_TABLE = {&&INSTRUCTION_1, &&INSTRUCTION_2,  
...};
```

```
goto *DISPATCH_TABLE[*ip];  
INSTRUCTION_1:  
    // Subroutines.  
    ip++;  
    goto *DISPATCH_TABLE[*ip];  
INSTRUCTION_2:  
    // Subroutines.  
    ip++;  
    goto *DISPATCH_TABLE[*ip];  
...
```

The current interpreter

CPython currently implements its interpreter using gotos/switch-cases:

```
void *DISPATCH_TABLE = {&&INSTRUCTION_1, &&INSTRUCTION_2,  
...};
```

```
goto *DISPATCH_TABLE[*ip];
```

```
INSTRUCTION_1:
```

```
    // Subroutines.
```

```
    ip++;
```

```
    goto *DISPATCH_TABLE[*ip];
```

```
INSTRUCTION_2:
```

```
    // Subroutines.
```

```
    ip++;
```

```
    goto *DISPATCH_TABLE[*ip];
```

```
...
```

The current interpreter

CPython currently implements its interpreter using gotos/switch-cases:

```
void *DISPATCH_TABLE = {&&INSTRUCTION_1, &&INSTRUCTION_2,  
...};
```

```
goto *DISPATCH_TABLE[*ip];
```

```
INSTRUCTION_1:
```

```
    // Subroutines.
```

```
    ip++;
```

```
    goto *DISPATCH_TABLE[*ip];
```

```
INSTRUCTION_2:
```

```
    // Subroutines.
```

```
    ip++;
```

```
    goto *DISPATCH_TABLE[*ip];
```

```
...
```

The current interpreter

CPython currently implements its interpreter using gotos/switch-cases:

```
void *DISPATCH_TABLE = {&&INSTRUCTION_1, &&INSTRUCTION_2,  
...};
```

```
goto *DISPATCH_TABLE[*ip];
```

```
INSTRUCTION_1:
```

```
    // Subroutines.
```

```
    ip++;
```

```
    goto *DISPATCH_TABLE[*ip];
```

```
INSTRUCTION_2:
```

```
    // Subroutines.
```

```
    ip++;
```

```
    goto *DISPATCH_TABLE[*ip];
```

```
...
```

The current interpreter

CPython currently implements its interpreter using gotos/switch-cases:

```
void *DISPATCH_TABLE = {&&INSTRUCTION_1, &&INSTRUCTION_2,  
...};
```

```
goto *DISPATCH_TABLE[*ip];
```

```
INSTRUCTION_1:
```

```
    // Subroutines.
```

```
    ip++;
```

```
    goto *DISPATCH_TABLE[*ip];
```

```
INSTRUCTION_2:
```

```
    // Subroutines.
```

```
    ip++;
```

```
    goto *DISPATCH_TABLE[*ip];
```

```
...
```

The current interpreter

CPython currently implements its interpreter using gotos/switch-cases:

```
void *DISPATCH_TABLE = {&&INSTRUCTION_1, &&INSTRUCTION_2,  
...};
```

```
goto *DISPATCH_TABLE[*ip];
```

```
INSTRUCTION_1:
```

```
    // Subroutines.
```

```
    ip++;
```

```
    goto *DISPATCH_TABLE[*ip];
```

```
INSTRUCTION_2:
```

```
    // Subroutines.
```

```
    ip++;
```

```
    goto *DISPATCH_TABLE[*ip];
```

```
...
```

The current interpreter

CPython currently implements its interpreter using gotos/switch-cases:

```
void *DISPATCH_TABLE = {&&INSTRUCTION_1, &&INSTRUCTION_2,  
...};
```

```
goto *DISPATCH_TABLE[*ip];
```

```
INSTRUCTION_1:
```

```
    // Subroutines.
```

```
    ip++;
```

```
    goto *DISPATCH_TABLE[*ip];
```

```
INSTRUCTION_2:
```

```
    // Subroutines.
```

```
    ip++;
```

```
    goto *DISPATCH_TABLE[*ip];
```

```
...
```

The current interpreter

CPython currently implements its interpreter using gotos/switch-cases:

```
void *DISPATCH_TABLE = {&&INSTRUCTION_1, &&INSTRUCTION_2,  
...};
```

```
goto *DISPATCH_TABLE[*ip];
```

```
INSTRUCTION_1:
```

```
    // Subroutines.
```

```
    ip++;
```

```
    goto *DISPATCH_TABLE[*ip];
```

```
INSTRUCTION_2:
```

```
    // Subroutines.
```

```
    ip++;
```

```
    goto *DISPATCH_TABLE[*ip];
```

```
...
```

One jump per instruction



The current interpreter

CPython currently implements its interpreter using gotos/switch-cases:

```
void *DISPATCH_TABLE = {&&INSTRUCTION_1, &&INSTRUCTION_2,  
...};
```

```
goto *DISPATCH_TABLE[*ip];
```

```
INSTRUCTION_1:
```

```
    // Subroutines.
```

```
    ip++;
```

```
    goto *DISPATCH_TABLE[*ip];
```

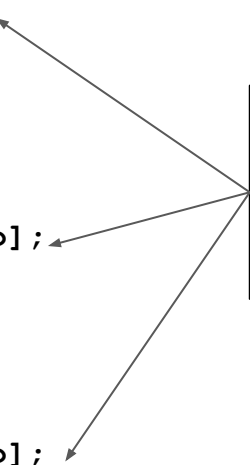
```
INSTRUCTION_2:
```

```
    // Subroutines.
```

```
    ip++;
```

```
    goto *DISPATCH_TABLE[*ip];
```

```
...
```



Jump targets spread out,
good for older branch
predictors

Problems With The Goto Interpreter

- No direct control over what to put into machine registers. Just trust the register allocator
- Massive — over 12000 lines of generated C code in Python 3.15 in a single function. Prone to compiler bugs due to its size
- Labels in labels in labels... lotsa unstructured control flow

Problems With The Switch & Goto Interpreters

Too complicated, easily breaks compiler heuristics

- <https://github.com/llvm/llvm-project/issues/106846> (Clang: performance regression in clang-19 when using computed goto)
- <https://github.com/python/cpython/issues/129987> (GCC: Autovectorization issues)
- https://gcc.gnu.org/bugzilla/show_bug.cgi?id=39284 (GCC: Computed gotos combined too aggressively)
- https://gcc.gnu.org/bugzilla/show_bug.cgi?id=71785 (GCC: Computed gotos are mostly optimized away)
- <https://github.com/python/cpython/issues/121263> (MSVC: Not inlining small functions in the interpreter loop)
- <https://github.com/python/cpython/issues/125217> (MSVC: Compiler crash when bulding with PGO+JIT)

Problems With The Goto Interpreter

Buggy (GCC 13-15beta):

```
747a: movzbl    %ah, %ebx
747d: movzbl    %al, %eax
7480: movl     %ebx, -0x248(%rbp)
7486: movq     (,%rax,8), %rax
748e: jmp      0x72a0
<_PyEval_EvalFrameDefault+0x970>

72a0: movq     %r15, %xmm0
72a5: movq     %r13, %xmm3
72aa: movq     %r15, %rbx
72ad: punpcklqdq %xmm3, %xmm0
72b1: movhps   %xmm0, %xmm2
72b4: movq     %xmm2, %r10
72b9: movq     %r10, %r11
72bc: jmpq     *%rax
```

Efficient (GCC 11):

```
738f: movq     %r13, %r15
7392: movzbl    %ah, %ebx
7395: movzbl    %al, %eax
7398: movq     (,%rax,8), %rax
73a0: movl     %ebx, -0x248(%rbp)
73a6: jmpq     *%rax
```

DISPATCH body in CPython interpreter:

Incorrect optimization might cause important variables to get shuffled in and out of machine registers.

The search for a better construct

```
int foo_goto(n) {
```

```
  start:
```

```
    if (n <= 1) return 1;
```

```
    n--;
```

```
    goto start;
```

```
}
```

≈

```
int foo_tailcall(n) {
```

```
    if (n <= 1) return 1;
```

```
    [[musttail]]
```

```
    return foo(n-1);
```

```
}
```

The search for a better construct

```
int foo_goto(n) {  
  start:  
    if (n <= 1) return 1;  
    n--;  
    goto start;  
}  
  
≈  
  
int foo_tailcall(n) {  
    if (n <= 1) return 1;  
    [[musttail]]  
    return foo(n-1);  
}
```

1977: Concept observed by Guy Steele

The search for a better construct

```
int foo_goto(n) {  
    start:  
        if (n <= 1) return 1;  
        n--;  
        goto start;  
}  
  
≈  
  
int foo_tailcall(n) {  
    if (n <= 1) return 1;  
    [[musttail]]  
    return foo(n-1);  
}
```

1977: Concept observed by Guy Steele
2000-ish: Tail call optimization added to GCC for C

The search for a better construct

```
int foo_goto(n) {  
    start:  
        if (n <= 1) return 1;  
        n--;  
        goto start;  
}  
  
≈  
  
int foo_tailcall(n) {  
    if (n <= 1) return 1;  
    [[musttail]]  
    return foo(n-1);  
}
```

1977: Concept observed by Guy Steele

2000-ish: Tail call optimization added to GCC for C

2021: [[musttail]] keyword added to Clang frontend

The search for a better construct

```
int foo_goto(n) {
```

```
start:
```

```
    if (n <= 1) return 1;
```

```
    n--;
```

```
    goto start;
```

```
}
```

≈

```
int foo_tailcall(n) {
```

```
    if (n <= 1) return 1;
```

```
    [[musttail]]
```

```
    return foo(n-1);
```

```
}
```

1977: Concept observed by Guy Steele

2000-ish: Tail call optimization added to GCC for C

2021: [[musttail]] keyword added to Clang frontend

2021: Protobuf adds an interpreter using [[musttail]] by Josh Haberman

The search for a better construct

```
int foo_goto(n) {  
    start:  
        if (n <= 1) return 1;  
        n--;  
        goto start;  
}  
  
≈  
  
int foo_tailcall(n) {  
    if (n <= 1) return 1;  
    [[musttail]]  
    return foo(n-1);  
}
```

1977: Concept observed by Guy Steele

2000-ish: Tail call optimization added to GCC for C

2021: [[musttail]] keyword added to Clang frontend

2021: Protobuf adds an interpreter using [[musttail]] by Josh Haberman

2025: [[musttail]] interpreter added to CPython

The search for a better construct

```
int foo_goto(n) {
```

```
start:
```

```
    if (n <= 1) return 1;
```

```
    n--;
```

```
    goto start;
```

```
}
```

≈

```
int foo_tailcall(n) {
```

```
    if (n <= 1) return 1;
```

```
    [[musttail]]
```

```
    return foo(n-1);
```

```
}
```

1977: Concept observed by Guy Steele

2000-ish: Tail call optimization added to GCC for C

2021: [[musttail]] keyword added to Clang frontend

2021: Protobuf adds an interpreter using [[musttail]] by Josh Haberman

2025: [[musttail]] interpreter added to CPython

2025-2026: [[musttail]] in GCC+MSVC

What is `musttail`?

- A tail call is a call expression in a return position in a function
- `musttail` guarantees that the call is converted to a *proper* tail call
- I.e. no function frames
- `musttail` is a compile-time guarantee, instead of an optimization-dependent one like tail call optimization

```
int foo_tailcall(n) {  
    if (n <= 1) return 1;  
    [[musttail]]  
    return foo(n-1);  
}
```

Goto

1 function, 12k+ lines of code

```
void *DISPATCH_TABLE = {&&INSTRUCTION_1,  
&&INSTRUCTION_2, ...};
```

```
goto *DISPATCH_TABLE[*ip];  
INSTRUCTION_1:  
    // Subroutines.  
    ip++;  
    goto *DISPATCH_TABLE[*ip];  
INSTRUCTION_2:  
    // Subroutines.  
    ip++;  
    goto *DISPATCH_TABLE[*ip];  
...
```

Tail Calls

225 functions, each up to 200 lines of code

```
funcptr[] DISPATCH_TABLE = {INSTRUCTION_1,  
INSTRUCTION_2, ...};
```

```
__attribute__((preserve_none))  
void INSTRUCTION_1(int *ip, ...) {  
    // Subroutines.  
    ip++;  
    [[musttail]]  
    return DISPATCH_TABLE[*ip](ip, ...);  
}
```

```
__attribute__((preserve_none))  
void INSTRUCTION_2(int *ip, ...) {  
    // Subroutines.  
    ip++;  
    [[musttail]]  
    return DISPATCH_TABLE[*ip](ip, ...);  
}  
...
```

Goto

1 function, 12k+ lines of code

```
void *DISPATCH_TABLE = {&&INSTRUCTION_1,  
&&INSTRUCTION_2, ...};
```

```
goto *DISPATCH_TABLE[*ip];  
INSTRUCTION_1:  
    // Subroutines.  
    ip++;  
    goto *DISPATCH_TABLE[*ip];  
INSTRUCTION_2:  
    // Subroutines.  
    ip++;  
    goto *DISPATCH_TABLE[*ip];  
...
```

Proper tail call attribute

Tail Calls

225 functions, each up to 200 lines of code

```
funcptr[] DISPATCH_TABLE = {INSTRUCTION_1,  
INSTRUCTION_2, ...};
```

```
__attribute__((preserve_none))  
void INSTRUCTION_1(int *ip, ...) {  
    // Subroutines.  
    ip++;  
    [[musttail]]  
    return DISPATCH_TABLE[*ip](ip, ...);  
}
```

```
__attribute__((preserve_none))  
void INSTRUCTION_2(int *ip, ...) {  
    // Subroutines.  
    ip++;  
    [[musttail]]  
    return DISPATCH_TABLE[*ip](ip, ...);  
}  
...
```

Goto

1 function, 12k+ lines of code

```
void *DISPATCH_TABLE = {&&INSTRUCTION_1,  
&&INSTRUCTION_2, ...};
```

```
goto *DISPATCH_TABLE[*ip];  
INSTRUCTION_1:  
    // Subroutines.  
    ip++;  
    goto *DISPATCH_TABLE[*ip];  
INSTRUCTION_2:  
    // Subroutines.  
    ip++;  
    goto *DISPATCH_TABLE[*ip];  
...
```

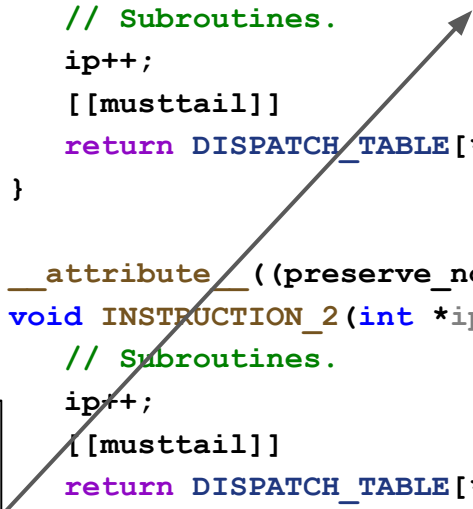
Special calling convention forces hottest state to always be in registers (with some caveats)

Tail Calls

225 functions, each up to 200 lines of code

```
funcptr[] DISPATCH_TABLE = {INSTRUCTION_1,  
INSTRUCTION_2, ...};
```

```
__attribute__((preserve_none))  
void INSTRUCTION_1(int *ip, ...) {  
    // Subroutines.  
    ip++;  
    [[musttail]]  
    return DISPATCH_TABLE[*ip](ip, ...);  
}  
  
__attribute__((preserve_none))  
void INSTRUCTION_2(int *ip, ...) {  
    // Subroutines.  
    ip++;  
    [[musttail]]  
    return DISPATCH_TABLE[*ip](ip, ...);  
}  
...
```



2%

Linux x86-64

5%

macOS Apple Silicon

15%

Windows MSVC x86-64

All numbers from pyperformance

Plus: All major compilers now support mandatory tail call attribute, and CPython became a rigorous test for `[[musttail]]` implementation

Why is this faster?

- On Windows MSVC: encourages compiler to inline functions, and encourages *direct threaded* code
- On other systems (and also on MSVC): better register allocation of hot variables due to `preserve_none`

preserve_none calling convention

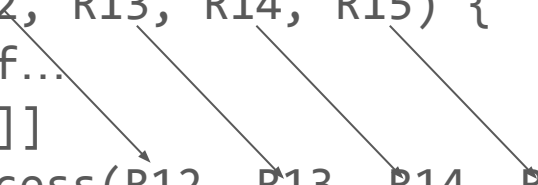
- Nearly all GPRs become caller-saved, except RSP and RBP on x86-64 in Clang 22.

```
__attribute__((preserve_none))
int process(R12, R13, R14, R15) {
    // Do stuff...
    [[musttail]]
    return process(R12, R13, R14, R15)
}
```

preserve_none calling convention

- Nearly all GPRs become caller-saved, except RSP and RBP on x86-64 in Clang 22.

```
__attribute__((preserve_none))
int process(R12, R13, R14, R15) {
    // Do stuff...
    [[musttail]]
    return process(R12, R13, R14, R15)
}
```



Keeps hot variables pinned

When is this *not* faster?

- Interpreters that make lots of calls to external code don't benefit as much (due to register shuffling from the `preserve_none` calling convention and system calling convention)
- Small-ish interpreters don't benefit as much, due to the compiler optimizing those better

Some `[[musttail]]` snags

Interaction with frame pointers (what's this? Backtraces? DWARF?):

- Naive `musttail/goto` implementation:
~2% perf hit.

PEP 831 – Frame Pointers Everywhere: Enabling System-Level Observability for Python

Author: Pablo Galindo Salgado <pablogsal at python.org>, Ken Jin <kenjin at python.org>, Savannah Ostrowski <savannah at python.org>, Diego Russo <diego.russo at arm.com>

Some `[[musttail]]` snags

Interaction with frame pointers (what's this? Backtraces? DWARF?):

- Naive `musttail/goto` implementation: ~2% perf hit.
- Reserve frame pointer register: ~0%
perf hit only on `musttail`

PEP 831 – Frame Pointers Everywhere: Enabling System-Level Observability for Python

Author: Pablo Galindo Salgado <pablogsal at python.org>, Ken Jin <kenjin at python.org>, Savannah Ostrowski <savannah at python.org>, Diego Russo <diego.russo at arm.com>

Some `[[musttail]]` snags

Interaction with frame pointers (what's this? Backtraces? DWARF?):

- Naive `musttail/goto` implementation: ~2% perf hit.
- Reserve frame pointer register: ~0%
perf hit only on `musttail`
- **I.e. with the right compiler flags, this makes frame pointers free for CPython's interpreter**

PEP 831 – Frame Pointers

Everywhere: Enabling System-Level Observability for Python

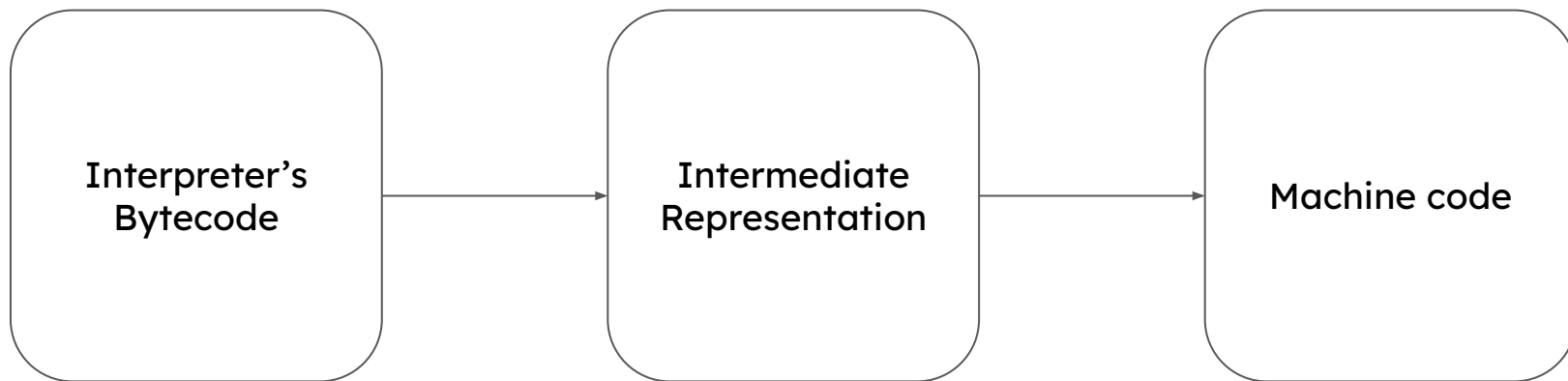
Author: Pablo Galindo Salgado <pablogsal at python.org>, Ken Jin <kenjin at python.org>, Savannah Ostrowski <savannah at python.org>, Diego Russo <diego.russo at arm.com>

Remaining problem

Interpretation still has overhead, we need compiled code to take over for hot paths

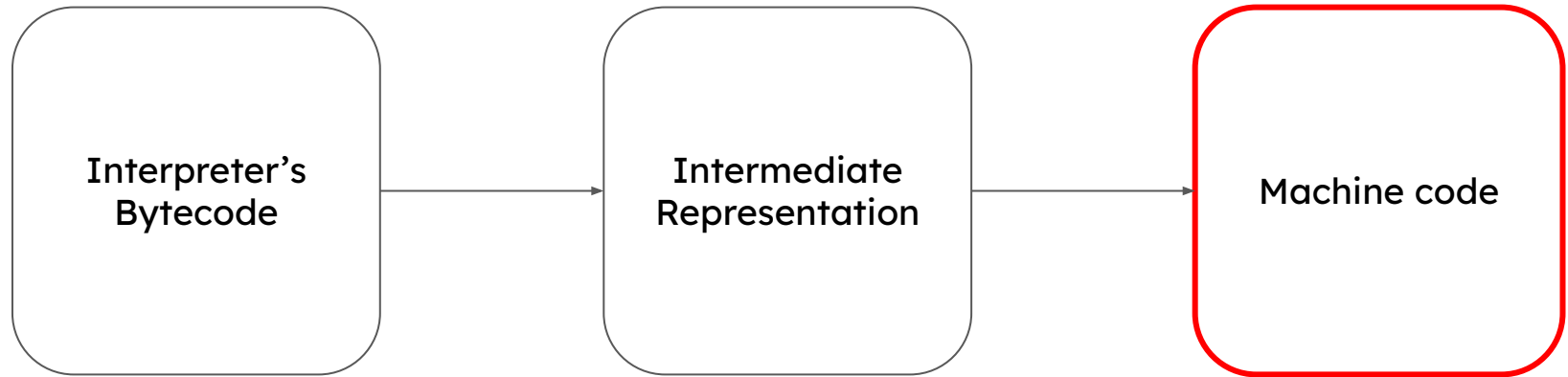
3.13-15: A simple JIT compiler

What's a JIT compiler?



All at runtime!

What's a JIT compiler?



All at runtime!

Problem

Writing a compiler backend to follow a spec is a ton of work.

How do you write codegen automatically?

Copy-and-Patch

By Haoran Xu, Fredrik Kjolstad in OOPSLA 2021

Copy-and-Patch

```
__attribute__((preserve_none))  
void INSTRUCTION_1(int *ip, ...) {  
    // Subroutines.  
    ip++;  
    [[clang::musttail]]  
    return DISPATCH_TABLE[*ip](ip, ...);  
}
```

Copy-and-Patch

```
__attribute__((preserve_none))  
void INSTRUCTION_1(int *ip, ...) {  
    // Subroutines.  
    ip++;  
    [[clang::musttail]]  
    return DISPATCH_TABLE[*ip](ip, ...);  
}
```

Observation: The interpreter is a definition of the language's semantics.

Copy-and-Patch

```
__attribute__((preserve_none))  
void INSTRUCTION_1(int *ip, ...) {  
    // Subroutines.  
    ip++;  
    [[clang::musttail]]  
    return DISPATCH_TABLE[*ip](ip, ...);  
}
```

Observation: The interpreter is a definition of the language's semantics.

If you stare hard enough, these `[[musttail]]` instructions are in the correct template form for compilation!

Copy-and-Patch

```
__attribute__((preserve_none))  
void INSTRUCTION_1(int *ip, ...) {  
    // Subroutines.  
    ip++;  
    [[clang::musttail]]  
    return DISPATCH_TABLE[*ip](ip, ...);  
}
```

If you stare hard enough, these `[[musttail]]` instructions are in the correct template form for compilation!

Interpreter overhead
(setup/teardown/traffic)

Observation: The interpreter is a definition of the language's semantics.

Copy-and-Patch

```
__attribute__((preserve_none))  
void INSTRUCTION_1(int *ip, ...) {  
    // Subroutines.  
    ip++;  
    [[clang::musttail]]  
    return DISPATCH_TABLE[*ip](ip, ...);  
}
```

If you stare hard enough, these `[[musttail]]` instructions are in the correct template form for compilation!

Interpreter overhead
(setup/teardown/traffic)

The actual language's semantics. Use this!

Observation: The interpreter is a definition of the language's semantics.

Copy-and-Patch

```
__attribute__((preserve_none))  
void INSTRUCTION_1(int *ip, ...) {  
    // Subroutines.  
    ip++;  
    [[clang::musttail]]  
    return DISPATCH_TABLE[*ip](ip, ...);  
}
```

Observation: The interpreter is a definition of the language's semantics.

If you stare hard enough, these `[[musttail]]` instructions are in the correct template form for compilation!

Interpreter overhead
(setup/teardown/traffic)

The actual language's semantics. Use this!

Represent runtime values as extern to tell the compiler to leave them as “holes” to burn in later.

Copy-and-Patch

```
__attribute__((preserve_none))  
void INSTRUCTION_1(int *ip, ...) {  
    // Subroutines.  
    ip++;  
    [[clang::musttail]]  
    return DISPATCH_TABLE[*ip](ip, ...);  
}
```

Observation: The interpreter is a definition of the language's semantics.

If you stare hard enough, these `[[musttail]]` instructions are in the correct template form for compilation!

Interpreter overhead
(setup/teardown/traffic)

The actual language's semantics. Use this!

Represent runtime values as extern to tell the compiler to leave them as “holes” to burn in later.

Not as easy to do with computed gotos.

Copy-and-Patch

```
// Subroutines.
```

Copy-and-Patch

```
// 0xabcd (LOAD_TEMPLATE)
```

Copy-and-Patch

```
// 0xabcd (LOAD_TEMPLATE)
```

```
// 0xbdef (LOAD_TEMPLATE)
```

```
// 0xdeed (ADD_TEMPLATE)
```

```
// 0xefef (STORE_TEMPLATE)
```

Copy-and-Patch

```
// 0xabcd (LOAD_TEMPLATE)
```

```
// 0xbdef (LOAD_TEMPLATE)
```

```
// 0xdeed (ADD_TEMPLATE)
```

```
// 0xefef (STORE_TEMPLATE)
```

Copy-and-Patch

```
// 0xabcd (LOAD)
```

```
// 0xbdef (LOAD_TEMPLATE)
```

```
// 0xdeed (ADD_TEMPLATE)
```

```
// 0xefef (STORE_TEMPLATE)
```

Copy-and-Patch

```
// 0xabcd (LOAD)
```

```
// 0xbdef (LOAD)
```

```
// 0xdeed (ADD_TEMPLATE)
```

```
// 0xefef (STORE_TEMPLATE)
```

Copy-and-Patch

```
// 0xabcd (LOAD)
```

```
// 0xbdef (LOAD)
```

```
// 0xdeed (ADD)
```

```
// 0xefef (STORE_TEMPLATE)
```

Copy-and-Patch

```
// 0xabcd (LOAD)
```

```
// 0xbdef (LOAD)
```

```
// 0xdeed (ADD)
```

```
// 0xefef (STORE)
```

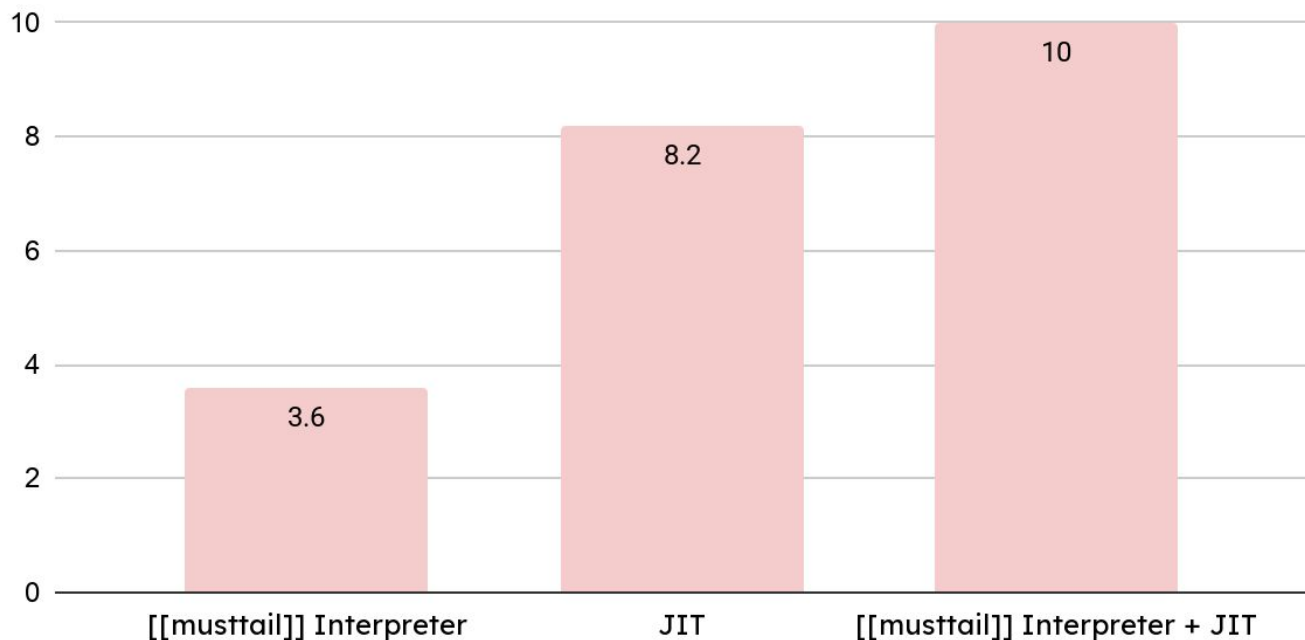
Copy-and-Patch

```
// 0xabcd (LOAD)
// 0xbdef (LOAD)
// 0xdeed (ADD)
// 0xefef (STORE)
```

= Decent, runnable, machine code

Tailcall+JIT = Efficiency Improvements

CPython 3.15 FastMark Geomean % Speedup Over
Computed Goto Interpreter Baseline



Setup:
CPython
3.15rc2

Clang 23

Ubuntu
26.04
x86_64

Frame
pointers
disabled

Noise: $\pm 1\%$

The Remaining Problem

Compiler	[[musttail]] semantics	preserve_none semantics
Clang 21	Variable lifetimes dropped at tail call	Explicitly not stable (probably changed since then)
GCC 15-16	Variable lifetimes dropped at tail call, but a warning appears	?
MSVC 2026 v145	Variable lifetimes not dropped at tail call. Possibly escaping variables are a compiler error.	?

The Remaining Problem

Compiler	[[musttail]] semantics	preserve_none semantics
Clang 21	Variable lifetimes dropped at tail call.	Explicitly not stable (probably changed since then)
GCC 15-16	Variable lifetimes dropped at tail call, but a warning appears	?
MSVC 2026 v14.5	Variable lifetimes not dropped at tail call. Possibly escaping variables are a compiler error.	?

We need the major implementations to agree!

A `[[musttail]]` Future

Draft C++29 proposal:

Special keyword at the return site (not yet 100% decided) to imply `[[musttail]]`

Rust nightly already has a similar keyword

Number: D3939R0

Date: 2025-12-09

Audience: [Evolution](#)

Target: C++29

Author: [Hana Dusíková](#)

- [Introduction](#)
- [Motivation](#)
 - [Security & safety](#)
 - [Crash in optimized build due to avoidable stack overflow](#)
- [Portability](#)
 - [What modern platform don't support tail-recursion call optimization?](#)
- [Constant evaluator using same code path](#)
- [Syntax options](#)
 - [Keyword](#)
 - [Annotation](#)
- [Design & behavior](#)
 - [Lifetime](#)
 - [Attribute or annotation on a call without return](#)
- [Implementation](#)
 - [Constant implementation](#)
- [Wording](#)
 - [Feature test macro](#)

tail recursion enforcement

This paper propose standardization of an attribute (or other mechanism) to enforce tail recursion on a return expression, to make this type of code portable and usable for unoptimized builds and also in constant evaluator.

As the author of this paper I don't dwell on any particular syntax, I present existing syntax in existing compilers and show its options. I care about the functionality guarantee of limiting the system stack size.

Introduction

Attribute `[[clang::musttail]]` in clang, `[[gnu::musttail]]` in GCC, and `[[msvc::musttail]]` in MSVC are **already existing practice** to enforce compiler emitting tail recursion.

While C++ doesn't define notion of a system stack or recursion limits, generally a program can't have arbitrary deep recursion without running into portability problems.

In practice compilers provide a hard limit for the recursion depth only in the constant evaluator (in clang the option is set with `-fconstexpr-depth`, with default value [512](#)).

Motivation

```
struct interpreter {
    // ...
    using instruction_type = void(*)(interpreter &);
    using symbol_type = ...;
    using stack_type = std::stack<symbol_type>;

    stack_type stack{};

    symbol_type read_symbol();
    instruction_type read_instruction();

    constexpr void execute() {
```

DRAFT